

# Argo

## Manual técnico do projeto

E-commerce em microserviços com Python, Django, Celery e Java

Copyright (©) 2026 Erik Pimenta

Versão 1.1.0 · Outubro de 2026 · Licença MIT

# Sumário

- **Visão geral** 4
  - Para que serve 4
  - O que o Argo faz 4
  - Tecnologias 5
  - Identidade visual 5
- **Como rodar** 6
  - Requisitos 6
  - Primeira execução 6
  - Comandos do dia a dia 7
  - Rodar sem Docker 7
- **Acesso e segurança** 8
  - Contas de demonstração 8
  - Papéis e permissões 8
  - Tokens e sessão 9
  - Proteções 9
- **Guia das telas** 10
  - Entrada 10
  - Visão geral 10
  - Loja 11
  - Carrinho 12
  - Pedido 13
  - Notas fiscais 14
  - Produtos 15
  - Estoque ao vivo 16
  - Pedidos (administrador) 17
  - Usuários 18
  - Documentação da API 19
- **Arquitetura** 21
  - Serviços 21
  - Como o Django está organizado 21

- Princípios 22
- Gateway 23
- Estrutura de pastas 23

**Processamento assíncrono** 24

- Tarefas 24
- Falhas e novas tentativas 25
- Situações do pedido 25
- Estoque em tempo real 25
- Monitoramento 25

**API** 27

- Primeiros passos 27
- Identidade · /api/identity 27
- Catálogo · /api/catalog 27
- Pedidos · /api/orders 28
- Faturamento · /api/billing 28
- Erros 29

**Configuração** 30

- Variáveis 30
- E-mail de verdade 30
- Checklist para uso próprio 31

**Testes e automação** 32

- Python 32
- Java 32
- Integração contínua 33
- Atividade diária 33

**Solução de problemas** 34

Capítulo 1

# Visão geral

O Argo é uma plataforma de vendas em microserviços, de código aberto. Vários vendedores cadastram produtos, o estoque é atualizado em tempo real em todas as telas e cada pedido é confirmado por uma fila de processamento, sem deixar o cliente esperando.

O projeto foi feito para ser usado, estudado e adaptado. Qualquer pessoa pode baixar o código, subir tudo com um comando e entrar com as contas de demonstração. Para uso próprio, basta trocar os segredos do arquivo `.env`, desligar os dados de demonstração e apontar o envio de e-mails para o seu servidor.

## Para que serve

Mostra, em um sistema funcionando de ponta a ponta, as peças que aparecem em qualquer e-commerce de verdade: autenticação com papéis, catálogo de vários vendedores, controle de estoque à prova de concorrência, processamento assíncrono com fila, envio de e-mail, emissão de nota fiscal por um serviço em outra linguagem e documentação completa da API.

## O que o Argo faz

### Para quem compra

- Loja com busca, filtro por categoria, ordenação e paginação; o saldo de cada produto muda na tela assim que alguém compra.
- Carrinho em gaveta lateral, limitado ao estoque disponível.
- Checkout que responde na hora e acompanhamento do pedido com etapas e linha do tempo atualizadas sozinhas.
- E-mail de confirmação, nota fiscal com o valor aproximado dos tributos e cancelamento enquanto o pedido não foi enviado.

### Para quem vende

- Painel com vendas por dia, ticket médio, mais vendidos e pedidos por situação, considerando só os produtos do vendedor.
- Cadastro de produtos com SKU, categoria, preço, estoque inicial e limite de estoque baixo.
- Movimentação de estoque (entrada, saída e contagem) com histórico imutável e tela de movimentações ao vivo.
- Pedidos que têm seus produtos, com marcação de enviado e entregue.

### Para quem administra

- Visão de toda a plataforma, gestão de usuários (papel, nome, ativação) e cancelamento de qualquer pedido.
- Desativar uma conta encerra na hora todas as sessões dela.

## Tecnologias

| Camada | Tecnologia | Papel |
|---|---|---|
| Serviços | Python 3.13 · Django 6 · Django REST Framework | Identidade, catálogo e pedidos, com OpenAPI 3.1 gerado pelo drf-spectacular |
| Persistência | ORM e migrações do Django · MySQL 8.4 | Um banco por serviço; SQLite nos testes |
| Servidor | Daphne (ASGI) | Rotas REST e a view assíncrona de tempo real no mesmo processo |
| Fila | Celery 5 · Redis 7 | Baixa de estoque, e-mail, nota fiscal e devoluções, configurado pelas settings do Django |
| Tempo real | Redis Pub/Sub · Server-Sent Events | Saldo de estoque enviado às telas abertas |
| Faturamento | Java 21 · Spring Boot 4 · JPA · Flyway | Emissão e cancelamento de notas fiscais |
| Segurança | JWT HS256 · bcrypt do Django · Spring Security | Tokens de acesso, refresh tokens e RBAC |
| Interface | HTML, CSS e JavaScript (módulos ES) | Painel sem etapa de compilação |
| Gateway | Nginx | Roteamento de `/api`, arquivos estáticos e SSE |
| E-mail | django.core.mail · SMTP · Mailpit | Envio real ou caixa de entrada de testes |
| Testes | pytest-django · pytest-cov · JUnit 5 | Unitários e de integração, cobertura mínima de 80% |
| Entrega | Docker Compose · GitHub Actions | 10 contêineres, CI e atividade diária |

## Identidade visual

A interface segue o mesmo padrão do projeto Velo: fundo cinza bem claro, superfícies brancas, bordas finas e um único tom de destaque, o verde-pinho, usado em gráficos, seleção e foco do teclado. A fonte é a Public Sans, com números tabulares em preços e totais.

| #F6F7F7 | #FFFFFF | #E2E5E6 | #5F666B | #15181B | #1F5C4C |
|---|---|---|---|---|---|
| Fundo das páginas | Cartões e tabelas | Bordas e divisórias | Textos de apoio | Texto e botões | Destaque e gráficos |

As animações são fluidas e sempre curtas: troca de tela com leve subida, entrada escalonada de cartões e linhas, números que contam até o valor, barras que crescem, gaveta e modais com mola suave e destaque no saldo que acabou de mudar. Quem prefere menos movimento no sistema operacional vê tudo sem animação.

A logo é um veleiro, a nau Argo, em branco sobre fundo preto, centralizada como nas logos dos outros projetos. Ela fica em `web/img/logo.svg` e em `docs/images/logo.png`.

Capítulo 2

# Como rodar

O caminho recomendado é o Docker Compose: um comando sobe os 10 contêineres, cria os bancos, aplica as migrações do Django e cadastra os dados de demonstração.

## Requisitos

| Item | Versão | Observação |
|---|---|---|
| Docker Desktop (Windows/macOS) ou Docker Engine (Linux) | 24 ou mais novo | Com o plugin `docker compose` |
| Memória livre | 3 GB | MySQL, Redis, 4 serviços Python, Java e Nginx |
| Portas livres | 8080, 8025, 5555, 3306 | Todas podem ser trocadas no `.env` |

## Primeira execução

### Windows

1. Instale e abra o Docker Desktop.
2. Dê dois cliques em `iniciar.cmd`.
3. Na primeira vez, o script cria o `.env` com senha do banco, segredo do JWT e token interno aleatórios.
4. Espere a mensagem **Pronto!**; o navegador abre em http://localhost:8080.

### Linux ou macOS

```
./iniciar.sh
```

### Sem os scripts

```
cp .env.example .env
# edite ARGO_DB_PASSWORD, ARGO_JWT_SECRET e ARGO_INTERNAL_TOKEN
docker compose up -d --build
```

| Endereço | O que é |
|---|---|
| http://localhost:8080 | Aplicação |
| http://localhost:8080/docs/ | Swagger dos quatro serviços |
| http://localhost:8025 | Mailpit: caixa de entrada dos e-mails de teste |
| http://localhost:5555 | Flower: tarefas da fila em tempo real |

## Comandos do dia a dia

| Comando | Para quê |
|---|---|
| `docker compose ps` | Ver a situação e a saúde de cada contêiner |
| `docker compose logs -f worker` | Acompanhar as tarefas da fila |
| `docker compose restart orders` | Reiniciar um serviço |
| `docker compose up -d --scale worker=3` | Rodar três workers dividindo a mesma fila |
| `docker compose exec orders python manage.py showmigrations` | Ver as migrações do Django aplicadas no banco do serviço |
| `docker compose down` | Parar tudo mantendo os dados |
| `docker compose down -v` | Parar e apagar os bancos (os dados de demonstração voltam na próxima subida) |

## Rodar sem Docker

Os três serviços Python são um único projeto Django. A variável `ARGO_SERVICE` escolhe qual serviço o processo vira, e `python manage.py serve` aplica as migrações, cria os dados de demonstração e sobe o Daphne. Rode MySQL e Redis locais (ou só esses dois pelo Compose: `docker compose up -d mysql redis mailpit`). No Linux e no macOS, o mysqlclient precisa da biblioteca cliente do MySQL (no Ubuntu: `sudo apt install pkg-config libmysqlclient-dev`); no Windows o pip já baixa o pacote pronto. Depois, suba cada processo em um terminal:

```
cd backend
python -m venv .venv && . .venv/bin/activate
pip install -r requirements-dev.txt
export ARGO_JWT_SECRET=um-segredo-longo ARGO_INTERNAL_TOKEN=outro-segredo
export ARGO_CATALOG_URL=http://localhost:8002/api/catalog ARGO_ORDERS_URL=http://localhost:8003/api/
orders
ARGO_SERVICE=identity ARGO_DATABASE_URL=mysql://argo:SENHA@localhost:3306/argo_identity python
manage.py serve --port 8001
ARGO_SERVICE=catalog  ARGO_DATABASE_URL=mysql://argo:SENHA@localhost:3306/argo_catalog  python
manage.py serve --port 8002
ARGO_SERVICE=orders   ARGO_DATABASE_URL=mysql://argo:SENHA@localhost:3306/argo_orders   python
manage.py serve --port 8003
ARGO_SERVICE=worker celery -A argo.worker.celery_app:celery worker -Q orders -l info
```

```
cd billing
ARGO_BILLING_DB_URL=jdbc:mysql://localhost:3306/argo_billing ARGO_DB_USER=argo
ARGO_DB_PASSWORD=SENHA mvn spring-boot:run
```

Sem `ARGO_DATABASE_URL`, cada serviço Python usa um arquivo SQLite local; sem `ARGO_BILLING_DB_URL`, o Java usa H2 em arquivo (modo MySQL). Para mexer só no Python, `ARGO_SERVICE=all python manage.py serve` sobe os três serviços em um processo, na porta 8000, cada um com o seu banco SQLite. Os esquemas OpenAPI ficam em `/api/identity/openapi.json`, `/api/catalog/openapi.json`, `/api/orders/openapi.json` e `:8080/api/billing/openapi.json`.

Capítulo 3

# Acesso e segurança

Toda tela e toda rota exigem login, exceto a própria tela de entrada, o cadastro e as verificações de saúde.

## Contas de demonstração

Todas usam a senha **Argo@2026**. Os nomes são fictícios e os e-mails usam o domínio de exemplo `argo.dev`; nenhum dado pessoal real está no repositório.

| Papel | E-mail | Acesso |
|---|---|---|
| Administrador | `admin@argo.dev` | Tudo, incluindo usuários |
| Vendedor | `vendedor@argo.dev` | Loja Aurora: 8 produtos, estoque e vendas |
| Vendedor | `atelie@argo.dev` | Ateliê Norte: 8 produtos |
| Cliente | `cliente@argo.dev` | Loja, pedidos e notas próprias |

Antes de publicar para uso real, suba com `ARGO_SEED=false` em bancos vazios ou troque as senhas dessas contas em **Minha conta**.

## Papéis e permissões

| Ação | Cliente | Vendedor | Administrador |
|---|---|---|---|
| Ver loja e produtos ativos | sim | sim | sim |
| Fechar pedido | sim | não | não |
| Ver pedidos | os próprios | com produtos dele | todos |
| Cancelar pedido | os próprios | não | todos |
| Marcar enviado / entregue | não | com produtos dele | todos |
| Cadastrar e editar produtos | não | os próprios | todos |
| Movimentar estoque | não | os próprios | todos |
| Ver notas fiscais | as próprias | com produtos dele | todas |
| Gerenciar usuários | não | não | sim |

As permissões são conferidas no servidor de cada serviço (classes de permissão do Django REST Framework, como `roles(Role.SELLER, Role.ADMIN)`, e `hasAnyRole` no Spring Security). A interface só esconde o que o papel não pode usar.

## Tokens e sessão

- **Access token:** JWT HS256 de 15 minutos com `sub`, `role`, `name`, `email`, `iss`, `iat`, `exp`, `jti` e `type=access`. Os quatro serviços validam assinatura, emissor, tipo e validade; nos serviços Django isso é a classe de autenticação do DRF, sem consulta ao banco.
- **Refresh token:** valor aleatório de 64 caracteres, de uso único, válido por 7 dias. O banco guarda só o SHA-256.
- **Rotação:** cada `POST /auth/refresh` devolve um par novo e invalida o token usado.
- **Detecção de reuso:** se um refresh token já trocado aparece de novo, toda a família de tokens daquela sessão é revogada (a revogação é gravada antes da resposta de erro).
- **Logout, troca de senha e desativação** revogam os refresh tokens da conta.
- **Na interface**, os tokens ficam só na memória da aba. Recarregar ou fechar a página exige novo login; 20 minutos sem clique, tecla ou rolagem também. O access token é renovado sozinho antes de expirar.

## Proteções

| Proteção | Como funciona |
|---|---|
| Senhas | Hasher bcrypt do Django (`BCryptSHA256PasswordHasher`, custo 12); mínimo de 8 caracteres com letras e números |
| Força bruta | 5 senhas erradas seguidas bloqueiam a conta por 5 minutos (HTTP 423) |
| Enumeração de contas | Mesma mensagem e tempo de resposta parecido para e-mail inexistente e senha errada |
| Rotas internas | `/internal/*` exigem `X-Internal-Token`, comparado em tempo constante |
| Concorrência | `select_for_update()` na baixa de estoque e versão otimista na edição de produtos (HTTP 409) |
| Segredos | Só no `.env`, ignorado pelo Git; os scripts geram valores aleatórios |
| Cabeçalhos | `X-Content-Type-Options`, `X-Frame-Options: DENY` no Django e no Nginx; `manage.py check --deploy` roda no CI |
| Contêineres | Serviços Python e Java rodam com usuário sem privilégios; banco, Mailpit e Flower só escutam em 127.0.0.1 |

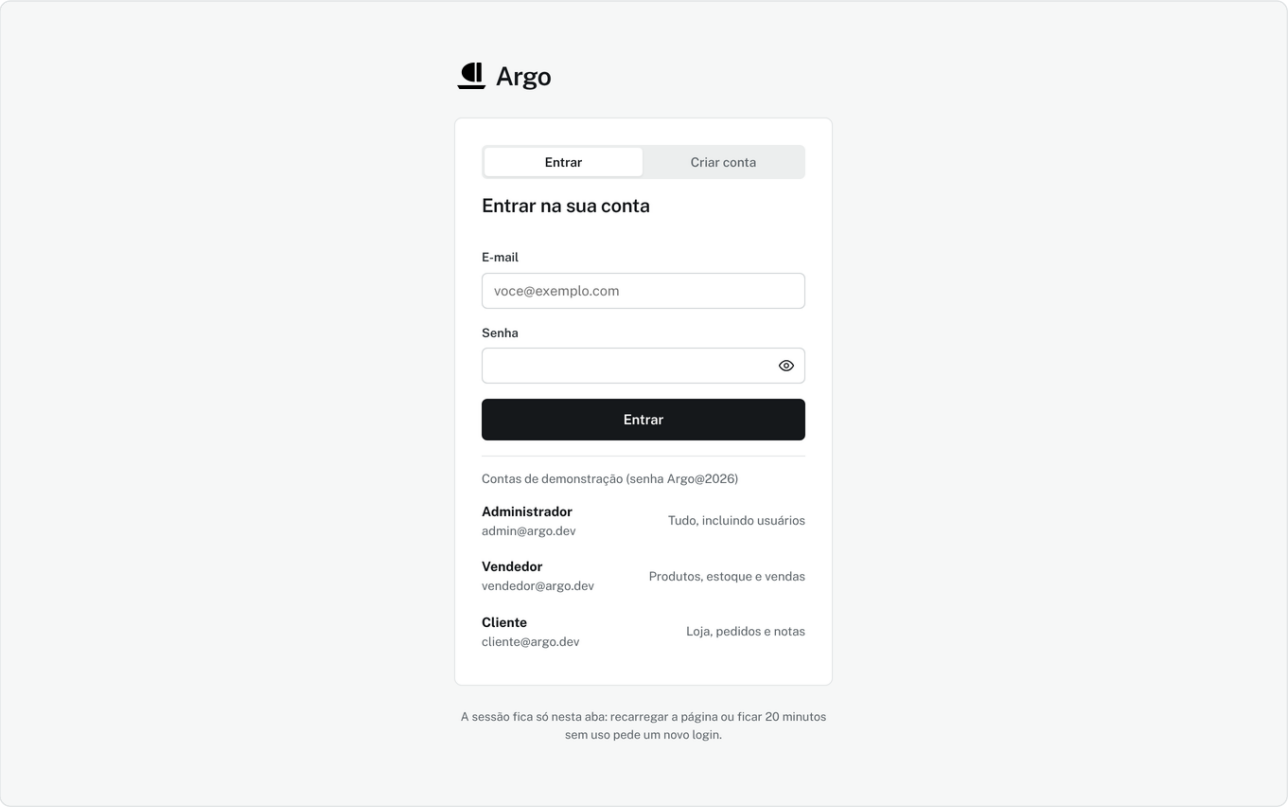
Capítulo 4

# Guia das telas

A barra lateral mostra só as telas do papel de quem entrou. O ponto verde no rodapé indica que a conexão de estoque em tempo real está ativa.

## Entrada

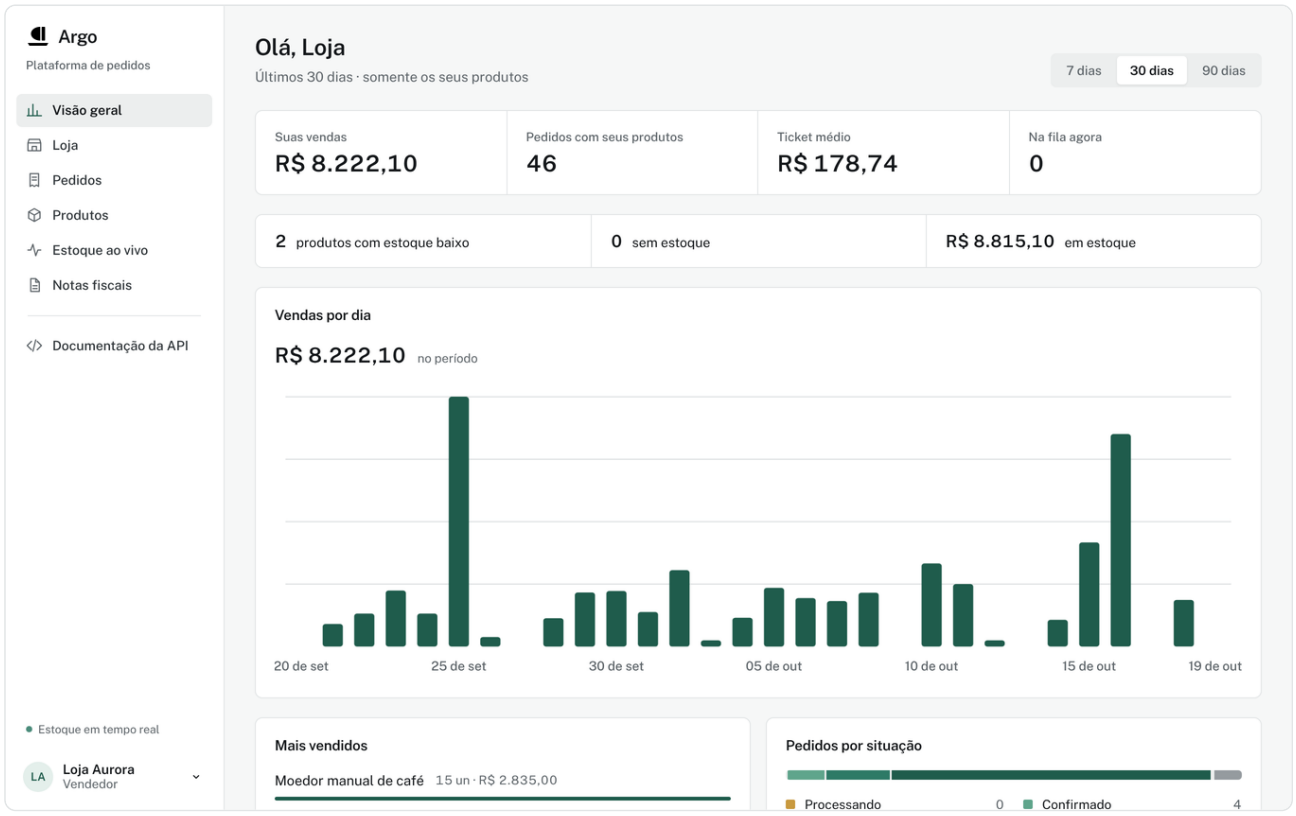
Abas para entrar ou criar conta (Cliente ou Vendedor). Clicar em uma conta de demonstração preenche e-mail e senha.




Entrada

## Visão geral

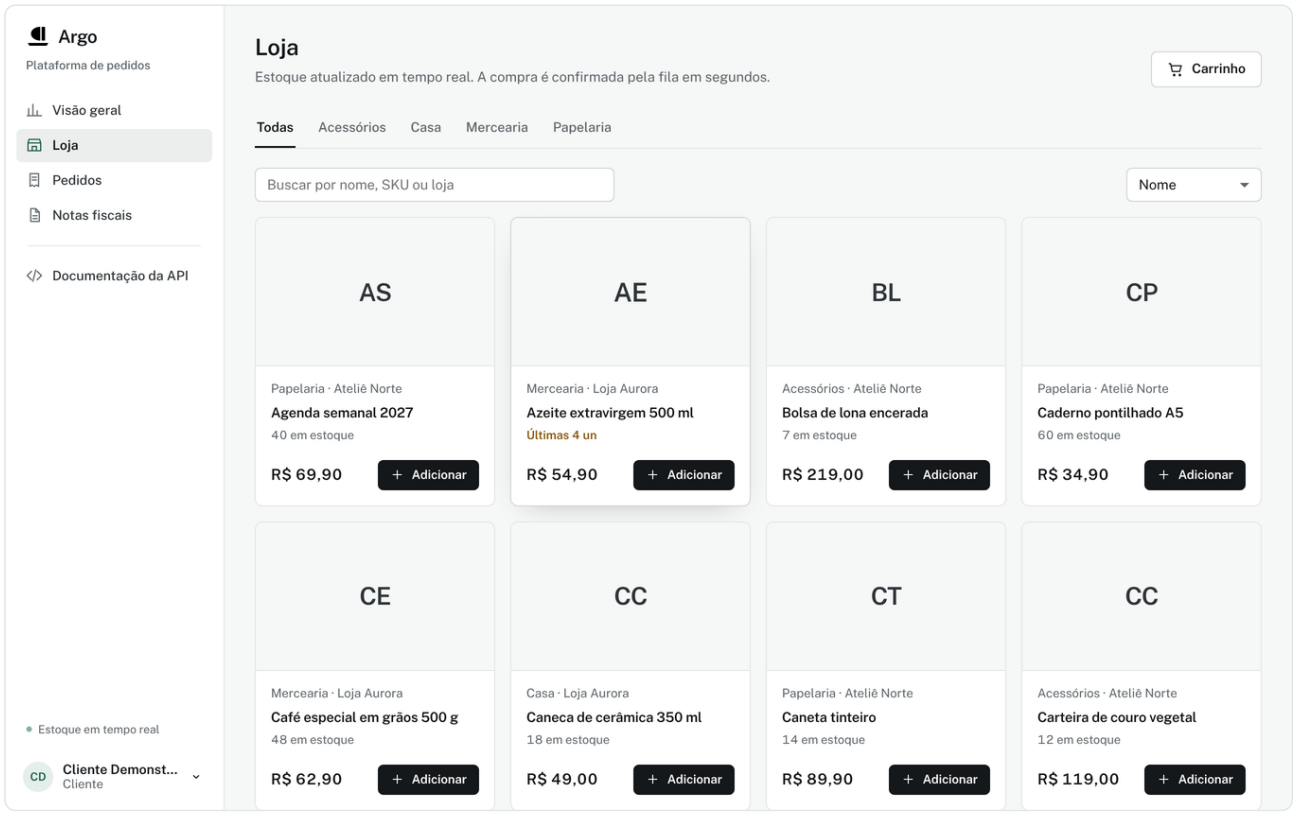
Indicadores do período (7, 30 ou 90 dias), itens que pedem atenção, vendas por dia (passe o mouse nas barras), mais vendidos, pedidos por situação e pedidos recentes. Cada papel vê só o que lhe cabe.




Visão geral

## Loja

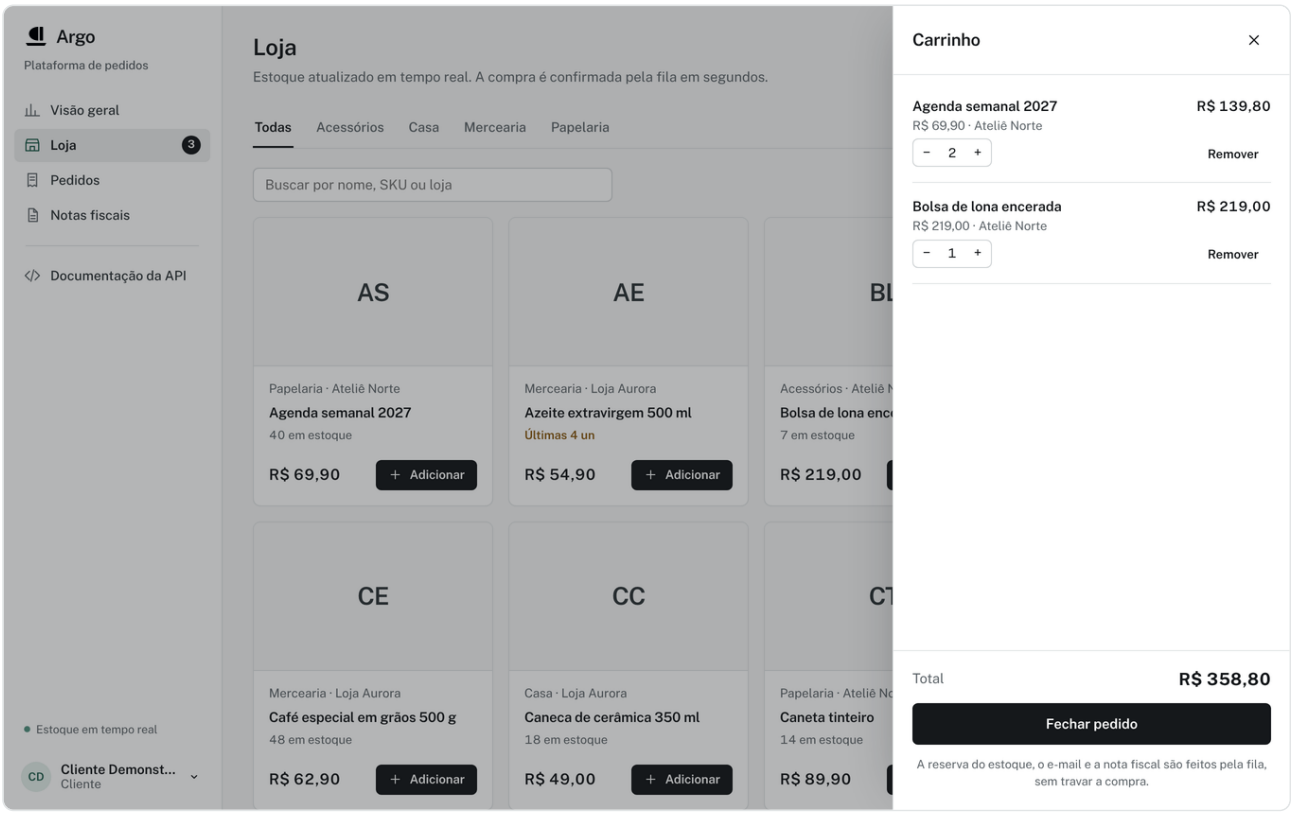
Categorias em abas, busca e ordenação. O saldo de cada produto muda na hora quando alguém compra, e o carrinho se ajusta se o estoque acabar.




Loja

## Carrinho

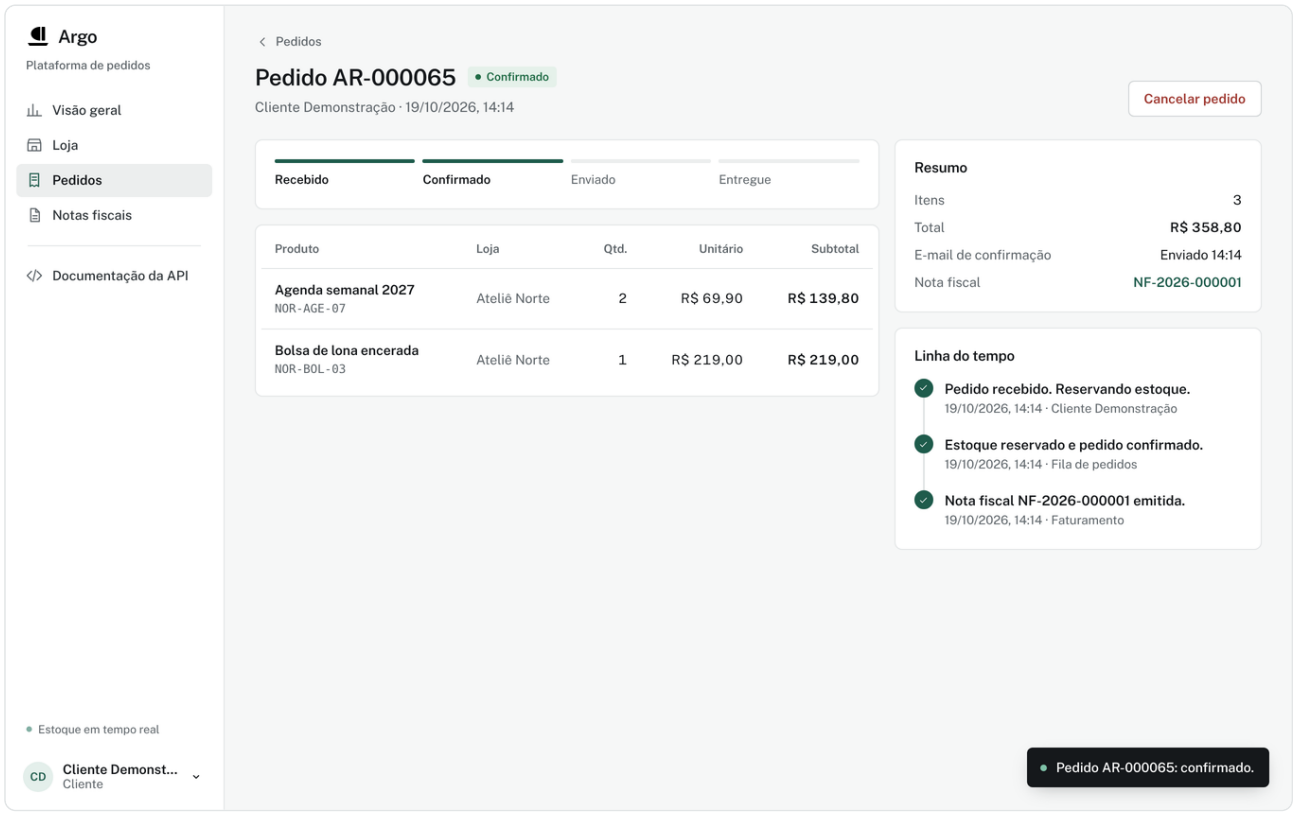
Gaveta lateral com quantidade limitada ao estoque. **Fechar pedido** envia para a fila e abre a página do pedido.




Carrinho

## Pedido

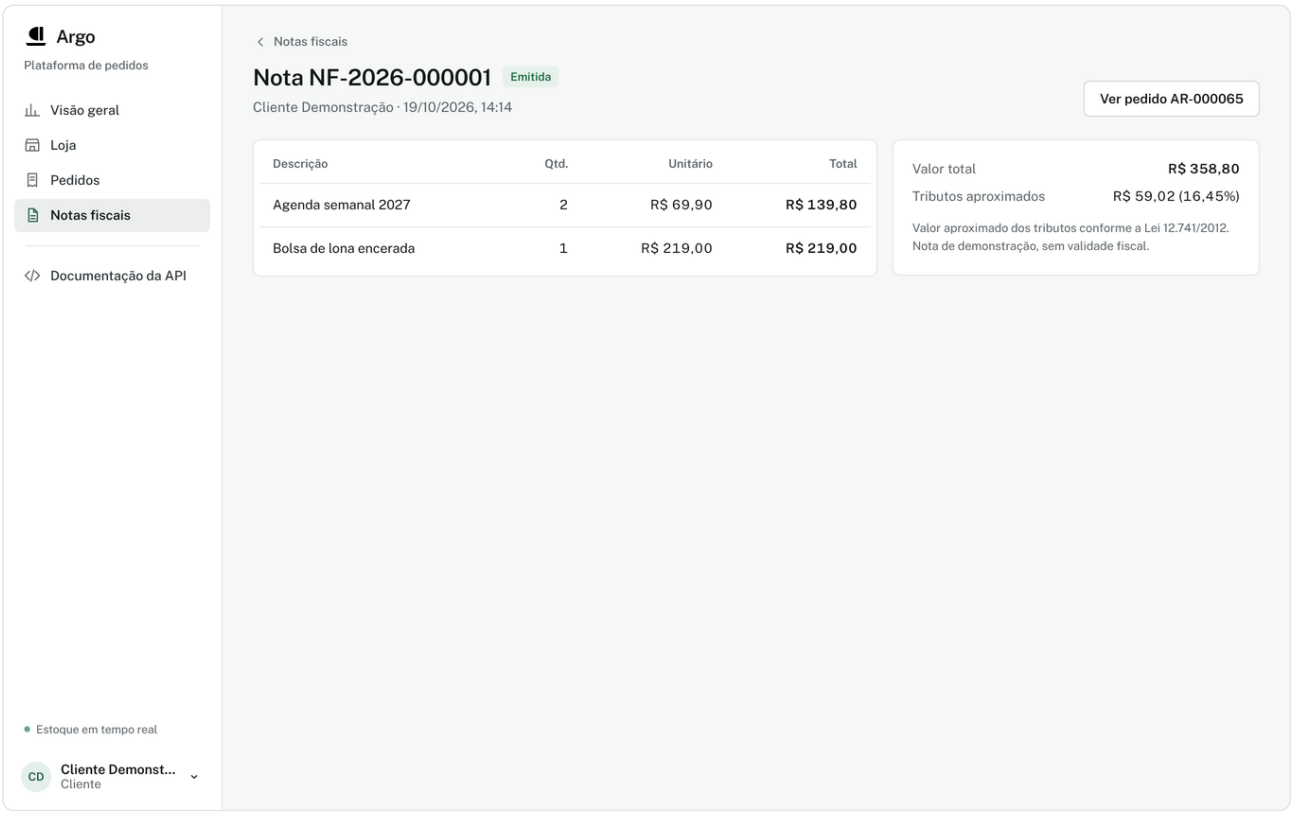
Etapas, itens, resumo e linha do tempo. Enquanto o pedido está na fila, a página se atualiza sozinha até mostrar a confirmação, o e-mail e a nota.




Pedido

## Notas fiscais

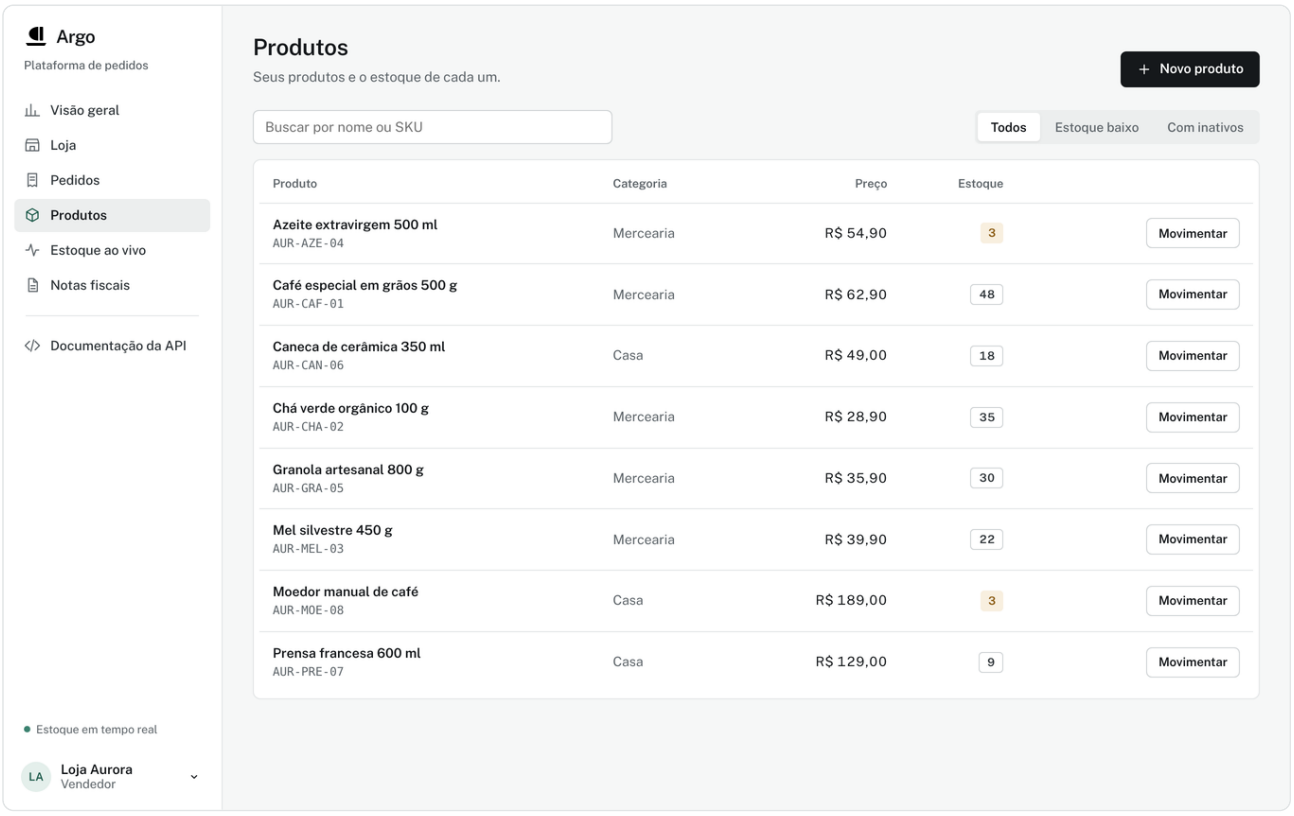
Nota emitida pelo serviço Java, com itens, total e valor aproximado dos tributos.




Notas fiscais

## Produtos

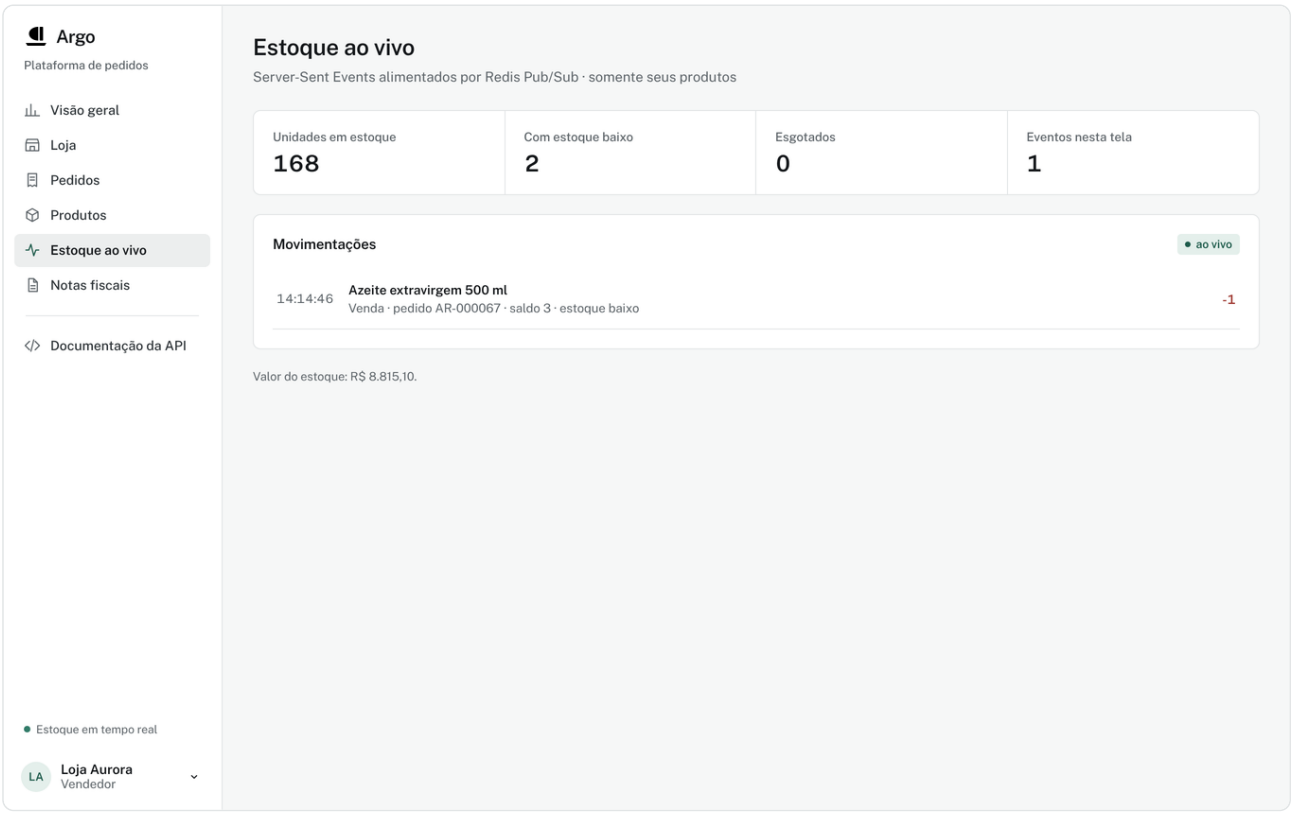
Lista com estoque destacado (amarelo no limite, vermelho esgotado), filtros e o botão **Movimentar** para entrada, saída ou contagem.




Produtos

## Estoque ao vivo

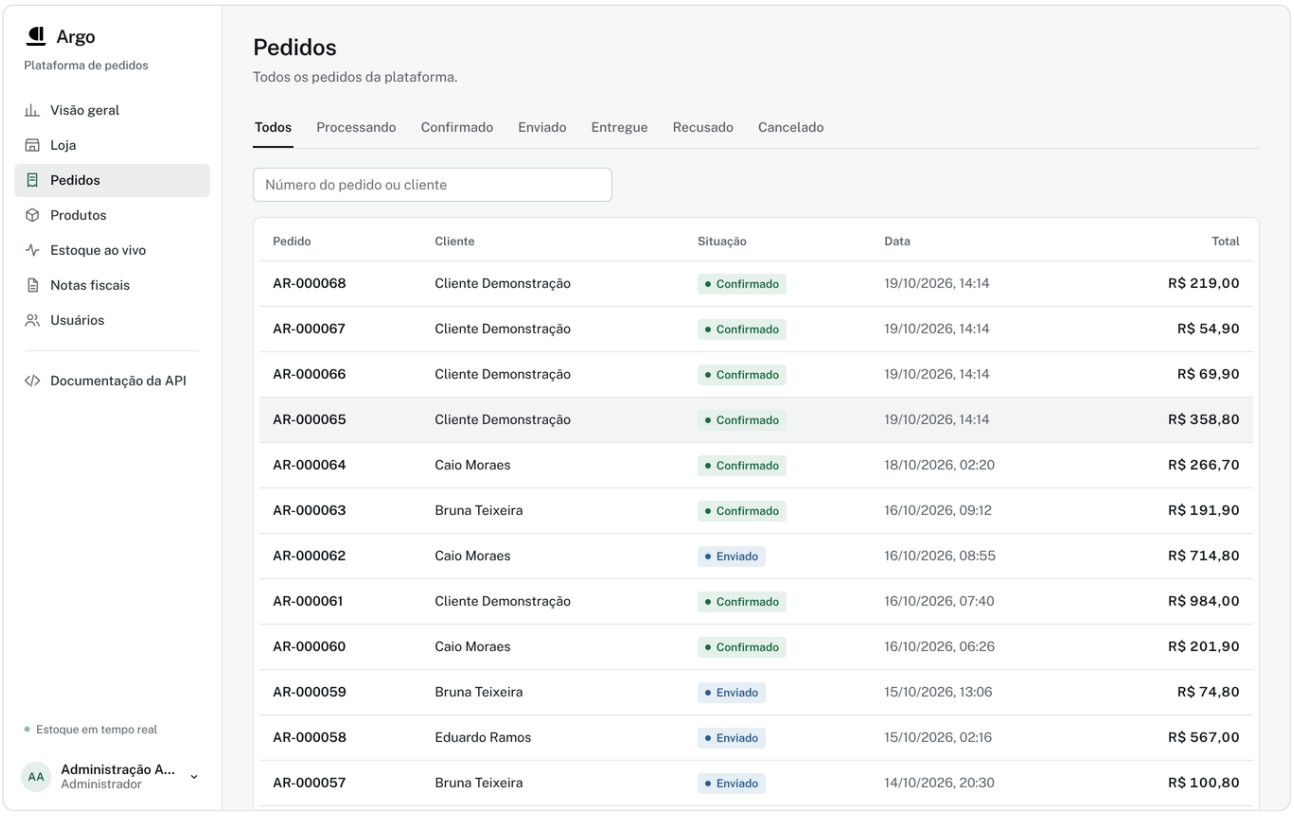
Cada venda, cancelamento ou ajuste aparece na hora, com o pedido de origem e o novo saldo.




Estoque ao vivo

## Pedidos (administrador)

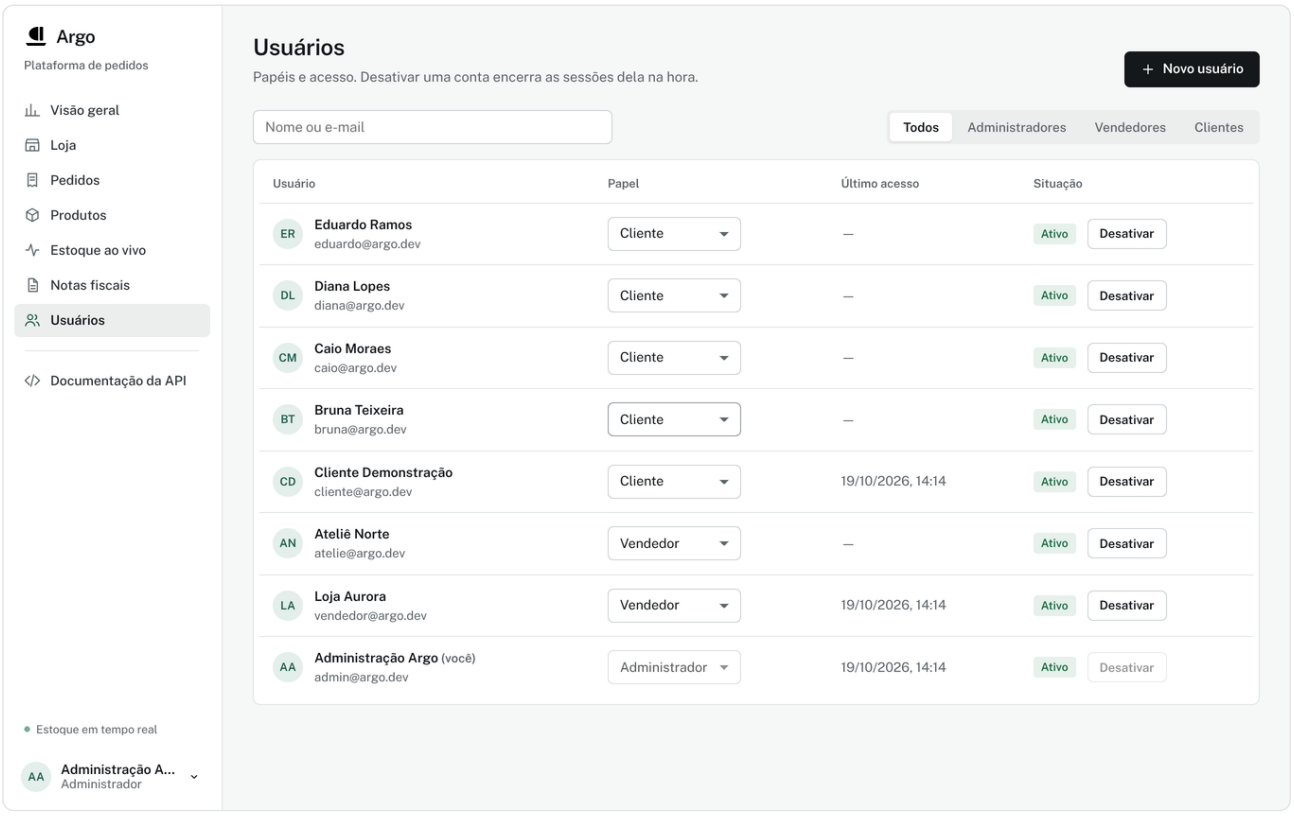
Filtro por situação e busca por número ou cliente.




Pedidos (administrador)

## Usuários

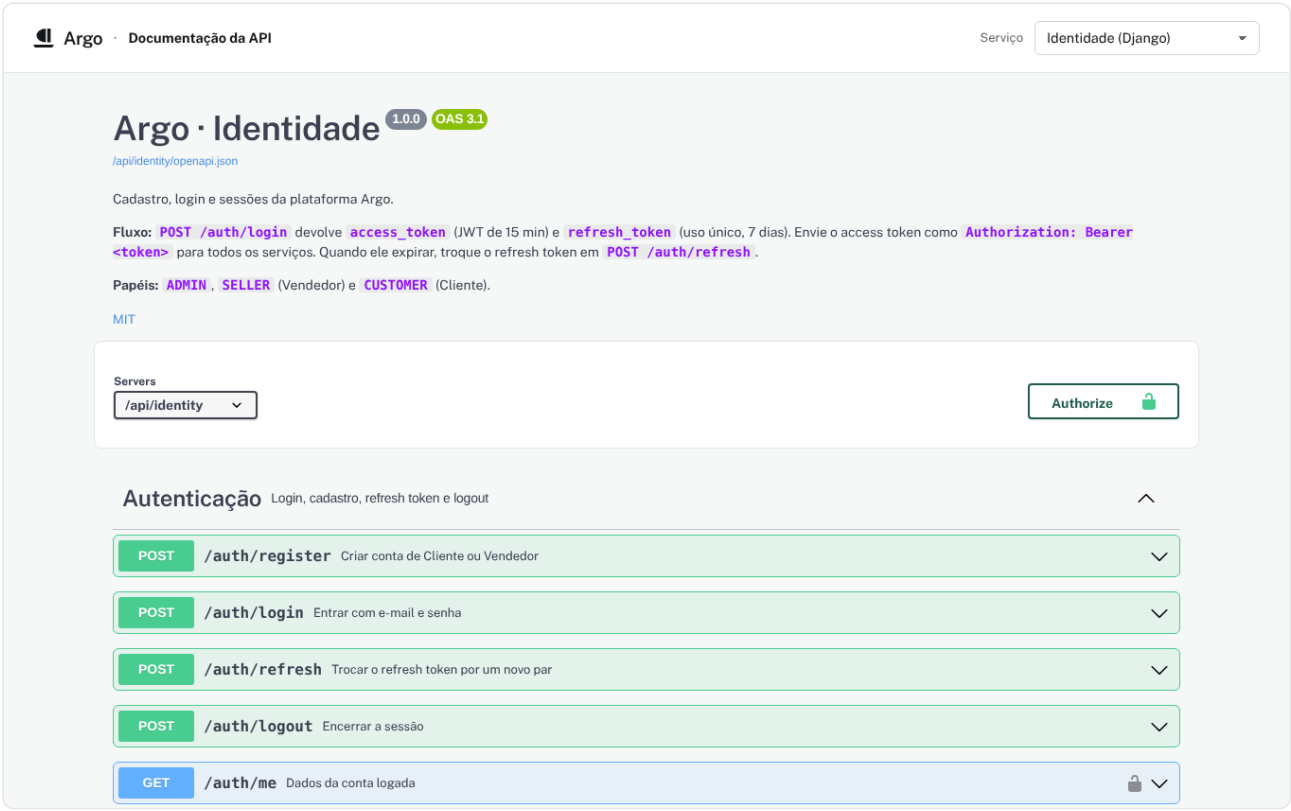
Troca de papel, desativação e criação de contas. O administrador não consegue remover o próprio acesso.




Usuários

## Documentação da API

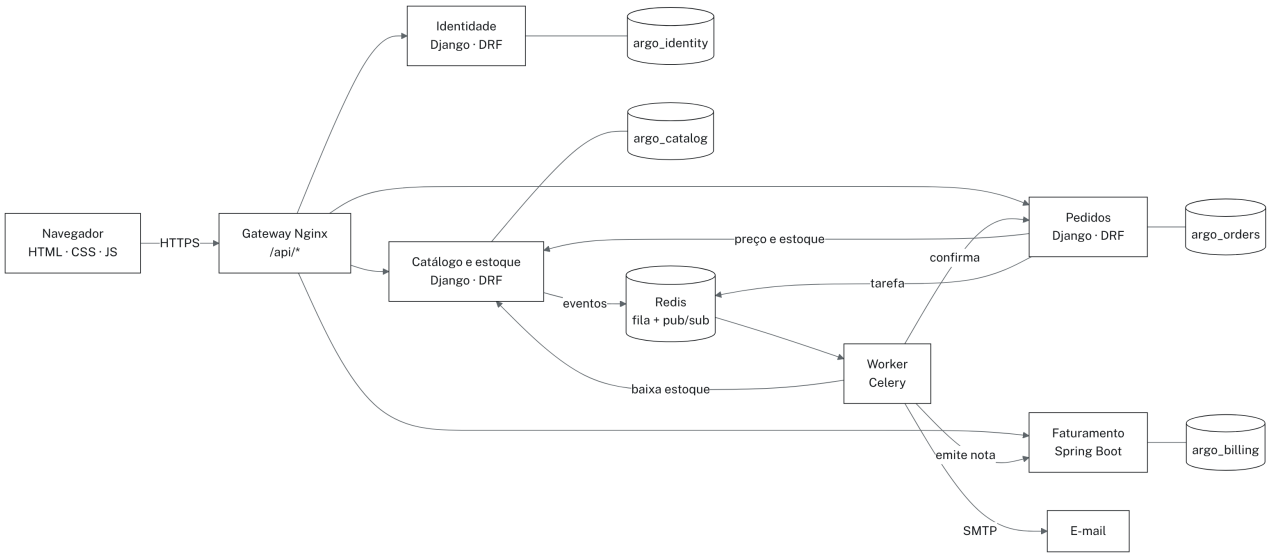
Swagger dos quatro serviços em uma página. Os serviços Django publicam o OpenAPI 3.1 gerado pelo drf-spectacular a partir das views e dos serializers; o Java publica o próprio arquivo openapi.json. Use **Authorize** com o access token para testar as rotas.




Documentação da API

Capítulo 5

# Arquitetura




Serviços, filas e bancos

## Serviços

| Serviço | Processo | Banco | Responsabilidade |
|---|---|---|---|
| `identity` | Django · Daphne :8000 | `argo_identity` | Contas, login, refresh tokens e usuários |
| `catalog` | Django · Daphne :8000 | `argo_catalog` | Produtos, estoque, movimentos e eventos |
| `orders` | Django · Daphne :8000 | `argo_orders` | Pedidos, situação e indicadores |
| `worker` | Celery | — | Tarefas da fila `orders` |
| `billing` | Spring Boot :8080 | `argo_billing` | Notas fiscais (Java) |
| `gateway` | Nginx 80 → 8080 | — | Interface, Swagger e roteamento |
| `mysql` · `redis` · `mailpit` · `flower` | — | — | Infraestrutura |

## Como o Django está organizado

Os três serviços Python são um único projeto Django em `backend/`. A variável `ARGO_SERVICE` decide o que o processo vira: ele instala só o app daquele serviço, liga só o banco dele e responde só em `/api/<serviço>/`. Assim o código comum é um só, mas cada serviço continua isolado em produção.

| Parte | Onde | O que faz |
|---|---|---|
| Configuração | `argo/settings.py` | Tudo vem das variáveis `ARGO_*`; `ARGO_SERVICE` = identity, catalog, orders, worker ou all |
| Modelos e migrações | `argo/<serviço>/models.py` · `migrations/` | Tabelas com CHECK, índices nomeados e `ON DELETE CASCADE` no próprio banco (`DB_CASCADE` do Django 6.1) |
| API REST | `views.py` · `serializers.py` · `urls.py` | Views do Django REST Framework, validação por serializers e rotas sem barra no fim |
| Autenticação | `argo/common/security.py` | Classe de autenticação JWT, permissões por papel e token interno |
| Erros | `argo/common/errors.py` | Exception handler do DRF que devolve sempre `{code, message, fields}` |
| Documentação | drf-spectacular | OpenAPI 3.1 por serviço em `/api/<serviço>/openapi.json` |
| Tempo real | `argo/catalog/views.py` | View assíncrona do Django que repassa o Redis Pub/Sub como SSE |
| Fila | `argo/worker/` · `argo/orders/dispatch.py` | Celery lendo as settings `CELERY_*`; envio só depois do COMMIT |
| E-mail | `argo/worker/mailer.py` | `django.core.mail` com o `MAILERS` do Django 6.1 |
| Comandos | `argo/common/management/commands/` | `serve` (migra, cria os dados e sobe o Daphne) e `seed` |
| Bancos | `argo/common/routers.py` | Roteador do Django: no modo `all` cada app grava no próprio banco |

## Princípios

- **Um banco por serviço.** Nenhum serviço consulta o banco de outro. O pedido guarda cópia do nome, preço e vendedor de cada item no momento da compra.
- **Comunicação explícita.** Leitura síncrona por HTTP com token interno e timeout curto; tudo que pode esperar vai para a fila.
- **Idempotência.** Restrições únicas no banco (um movimento de venda por pedido e produto, uma nota por pedido) garantem que mensagens repetidas não dupliquem nada.
- **Contrato único de erro** nos quatro serviços: `{code, message, fields}`.
- **Mesmo token em duas linguagens.** O serviço Java valida o JWT emitido pela identidade (Django) com o mesmo segredo, sem chamar a identidade.
- **Escala horizontal.** Serviços sem estado; os eventos de estoque passam pelo Redis, então qualquer réplica entrega o SSE.

## Gateway

| Caminho | Destino |
|---|---|
| `/` | Interface (arquivos de `web/`) |
| `/docs/` | Swagger UI dos quatro serviços |
| `/api/identity/*` | identity:8000, com o caminho inteiro (o Django já responde em /api/identity/) |
| `/api/catalog/*` | catalog:8000 (`/stock/stream` sem buffer, conexão de até 1 h) |
| `/api/orders/*` | orders:8000 |
| `/api/billing/*` | billing:8080 (o Spring usa o mesmo prefixo como context path) |
| `/api/<serviço>` | Redireciona para `/docs/?servico=<serviço>` |

## Estrutura de pastas

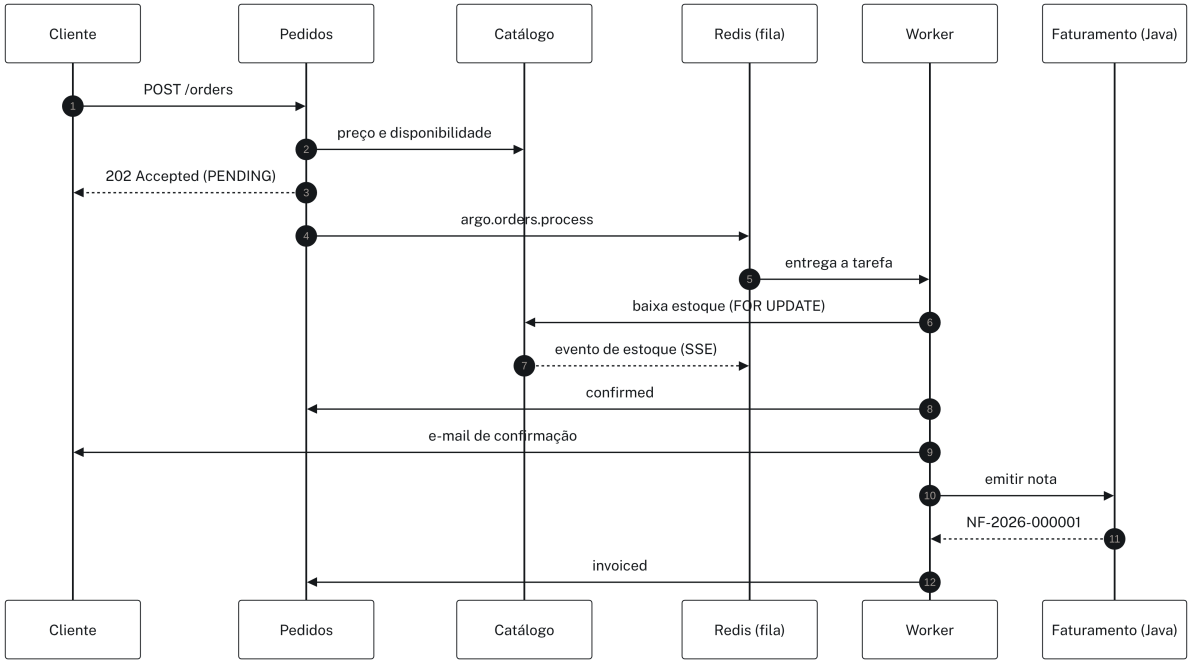
```
E-commerce-Microservices-Platform/
├── backend/manage.py              comandos do Django (migrate, serve, seed...)
├── backend/argo/settings.py       configuração e escolha do serviço (ARGO_SERVICE)
├── backend/argo/common/           JWT, permissões, erros, eventos, roteador, comandos
├── backend/argo/identity/         modelos, migrações, regras de sessão, views e rotas
├── backend/argo/catalog/          modelos, migrações, operações de estoque, SSE e rotas
├── backend/argo/orders/           modelos, migrações, dados de exemplo, fila e rotas
├── backend/argo/worker/           app Celery, tarefas e e-mail
├── backend/tests/                 65 testes (pytest-django)
├── billing/src/main/java/dev/argo/billing/
│     ├── invoice/                 entidades, repositório, serviço e controladores
│     ├── security/                validação do JWT, filtro e regras de acesso
│     └── common/ config/          erros, saúde, propriedades e relógio
├── billing/src/main/resources/db/migration/          Flyway
├── web/                           index.html, css/, js/ (páginas e componentes), docs/
├── gateway/                       nginx.conf
├── database/                      criação dos bancos e SQL gerado de cada serviço
├── scripts/                       gerar_esquema.py e criar-env.ps1
└── .github/workflows/             ci.yml e diario.yml
```

Capítulo 6

# Processamento assíncrono

O checkout não espera estoque, e-mail nem nota fiscal. O pedido é gravado como PENDING, a resposta sai com 202 Accepted e o resto acontece na fila do Celery, com Redis como broker.

A tarefa só é publicada depois do COMMIT ( `transaction.on_commit` ), então o worker nunca recebe um pedido que ainda não está no banco.




Do clique à nota fiscal

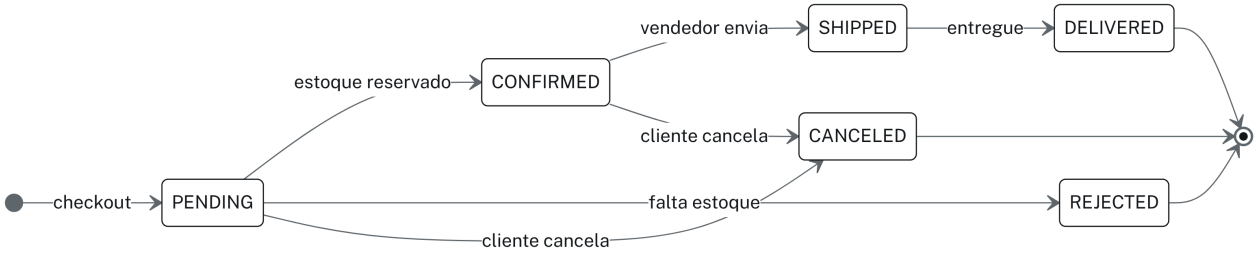
## Tarefas

| Tarefa | O que faz | Idempotência |
|---|---|---|
| `argo.orders.process` | Baixa o estoque do pedido inteiro (tudo ou nada) e confirma ou recusa | Ignora pedidos fora de PENDING; o catálogo não baixa duas vezes o mesmo pedido |
| `argo.orders.email` | Envia o e-mail de confirmação em HTML e texto | Só envia se `email_sent_at` estiver vazio |
| `argo.orders.invoice` | Pede a nota ao serviço Java e grava o número | Uma nota por pedido (restrição única no Java) |
| `argo.orders.cancel` | Devolve o estoque e cancela a nota | Movimento de devolução único por pedido e produto |

## Falhas e novas tentativas

- Erro de rede, HTTP 5xx, 408 ou 429 e falha de SMTP são temporários: a tarefa volta para a fila com espera exponencial (2 s, 4 s, 8 s… até 60 s, com variação aleatória), até 8 vezes.
- Erros 4xx são de regra e não são repetidos.
- `acks_late` e `reject_on_worker_lost`: se o worker cair no meio, a mensagem volta para a fila.
- Se o cliente cancelar entre a baixa do estoque e a confirmação, o worker percebe e devolve os itens.

## Situações do pedido




Ciclo de vida do pedido

| Situação | Na tela | Quem muda |
|---|---|---|
| `PENDING` | Processando | Checkout |
| `CONFIRMED` | Confirmado | Worker, depois de reservar o estoque |
| `REJECTED` | Recusado | Worker, quando falta estoque |
| `SHIPPED` | Enviado | Vendedor ou administrador |
| `DELIVERED` | Entregue | Vendedor ou administrador |
| `CANCELED` | Cancelado | Cliente ou administrador, enquanto PENDING ou CONFIRMED |

## Estoque em tempo real

Toda alteração de estoque publica, depois do COMMIT, um evento no canal `argo:stock` do Redis com produto, saldo, variação, tipo e se ficou abaixo do limite. A rota `GET /stock/stream` é uma view assíncrona do Django servida pelo Daphne (ASGI): mantém a conexão aberta e repassa cada evento como Server-Sent Event, com um comentário de batimento a cada 15 segundos sem eventos. Quando o navegador fecha a conexão, o Django cancela o fluxo e a inscrição no Redis é encerrada. A interface lê o fluxo com `fetch` (para enviar o cabeçalho `Authorization`) e reconecta sozinha com espera crescente.

## Monitoramento

- **Flower** (http://localhost:5555): tarefas recebidas, concluídas, repetidas e com falha, por worker.
- **Mailpit** (http://localhost:8025): todos os e-mails enviados, com versão HTML e texto.

- **Logs:** cada requisição registra método, caminho, situação e tempo; o cabeçalho `X-Request-ID` é devolvido em toda resposta.
- **Saúde:** `/api/<serviço>/health` em cada serviço Python, usado pelos healthchecks do Compose.

Capítulo 7

# API

Todas as rotas passam pelo gateway em `/api/<serviço>`, respondem JSON e exigem `Authorization: Bearer <token>`, exceto login, cadastro, refresh, logout e saúde. A documentação interativa fica em **/docs/**.

## Primeiros passos

```
TOKEN=$(curl -s -X POST localhost:8080/api/identity/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"cliente@argo.dev","password":"Argo@2026"}' | jq -r .access_token)

curl -s "localhost:8080/api/catalog/products?category=Casa" -H "Authorization: Bearer $TOKEN"

curl -s -X POST localhost:8080/api/orders/orders -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"items":[{"product_id":"ID","quantity":2}]}'
```

## Identidade · /api/identity

| Rota | Acesso | Descrição |
|---|---|---|
| `POST /auth/register` | público | Cria Cliente ou Vendedor e já devolve os tokens |
| `POST /auth/login` | público | Devolve access e refresh token |
| `POST /auth/refresh` | público | Troca o refresh token (rotação) |
| `POST /auth/logout` | público | Revoga a família do refresh token |
| `GET /auth/me` | logado | Dados da conta |
| `PUT /auth/me/password` | logado | Troca a senha e encerra as sessões |
| `GET /users` · `POST /users` · `PATCH /users/{id}` | ADMIN | Lista, cria e altera papel, nome ou situação |

## Catálogo · /api/catalog

| Rota | Acesso | Descrição |
|---|---|---|
| `GET /products` | logado | Busca, categoria, vendedor, `mine`, estoque baixo, ordenação e página |
| `GET /products/categories` | logado | Categorias com quantidade de produtos |
| `GET /products/{id}` | logado | Detalhe |

| Rota | Acesso | Descrição |
|---|---|---|
| `POST /products` | SELLER, ADMIN | Cadastra; estoque inicial vira movimento de entrada |
| `PUT /products/{id}` | dono, ADMIN | Edita; aceita `version` para concorrência otimista |
| `DELETE /products/{id}` | dono, ADMIN | Desativa (exclusão lógica) |
| `POST /products/{id}/stock` | dono, ADMIN | Entrada, saída ou contagem |
| `GET /products/{id}/movements` | dono, ADMIN | Histórico de estoque |
| `GET /stock/summary` | SELLER, ADMIN | Unidades, estoque baixo, esgotados e valor |
| `GET /stock/stream` | logado | Eventos de estoque (SSE) |

## Pedidos · /api/orders

| Rota | Acesso | Descrição |
|---|---|---|
| `POST /orders` | CUSTOMER | Checkout; responde 202 com o pedido em PENDING |
| `GET /orders` | logado | Lista por papel, com filtro de situação e busca |
| `GET /orders/stats?days=30` | logado | Faturamento, ticket médio, por dia, por situação e mais vendidos |
| `GET /orders/{id}` | logado | Pedido com itens e linha do tempo |
| `POST /orders/{id}/cancel` | dono, ADMIN | Cancela PENDING ou CONFIRMED |
| `PATCH /orders/{id}/status` | SELLER, ADMIN | SHIPPED ou DELIVERED |

## Faturamento · /api/billing

| Rota | Acesso | Descrição |
|---|---|---|
| `GET /invoices` | logado | Notas visíveis para o papel |
| `GET /invoices/summary` | logado | Emitidas, canceladas, valor e tributos |
| `GET /invoices/{número}` | logado | Nota com itens |
| `POST /internal/invoices` | interno | Emite (idempotente) |
| `POST /internal/invoices/{pedido}/cancel` | interno | Cancela |

## Erros

```
{
  "code": "validation_error",
  "message": "Confira os campos destacados.",
  "fields": { "price_cents": "Informe um valor maior ou igual a 1." }
}
```

| HTTP | Quando |
|---|---|
| 401 | Token ausente, inválido ou expirado |
| 403 | Papel sem permissão ou rota interna sem token |
| 404 | Não existe ou não pertence a quem pediu |
| 405 | Método não aceito na rota |
| 409 | Estoque insuficiente, SKU repetido, versão desatualizada ou situação inválida |
| 422 | Campos inválidos ou JSON malformado; erros de lista vêm como items.0, items.1... |
| 423 | Conta bloqueada por tentativas |
| 503 | Outro serviço não respondeu |

Capítulo 8

# Configuração

Todas as variáveis ficam no `.env`, criado a partir do `.env.example`. Os serviços Python leem o prefixo `ARGO_` nas settings do Django (`backend/argo/settings.py`); o Java lê as mesmas variáveis pelo `application.yml`.

## Variáveis

| Variável | Padrão | Uso |
|---|---|---|
| `ARGO_PORT` | 8080 | Porta da aplicação |
| `ARGO_PUBLIC_URL` | http://localhost:8080 | Endereço usado nos links do e-mail |
| `ARGO_DB_USER` / `ARGO_DB_PASSWORD` | argo / gerada | Usuário do MySQL |
| `ARGO_DB_PORT` | 3306 | Porta do MySQL no seu computador |
| `ARGO_JWT_SECRET` | gerado | Assina e valida os tokens (Django e Java); também é a SECRET_KEY do Django |
| `ARGO_INTERNAL_TOKEN` | gerado | Autentica as chamadas entre serviços |
| `ARGO_SEED` | true | Dados de demonstração em bancos vazios |
| `ARGO_BILLING_TAX_RATE` | 0.1645 | Alíquota aproximada exibida na nota |
| `ARGO_SMTP_HOST` / `PORT` / `USER` / `PASSWORD` / `TLS` | mailpit / 1025 | Servidor de e-mail |
| `ARGO_MAIL_FROM` | Argo <nao-responda@argo.dev> | Remetente |
| `ARGO_MAILPIT_PORT` / `ARGO_FLOWER_PORT` | 8025 / 5555 | Portas das ferramentas |

Variáveis só dos serviços (definidas no Compose): `ARGO_SERVICE` (identity, catalog, orders ou worker), `ARGO_DATABASE_URL`, `ARGO_REDIS_URL`, `ARGO_CATALOG_URL`, `ARGO_ORDERS_URL`, `ARGO_BILLING_URL`, `ARGO_ACCESS_TOKEN_MINUTES` (15), `ARGO_REFRESH_TOKEN_DAYS` (7), `ARGO_LOGIN_MAX_ATTEMPTS` (5), `ARGO_LOGIN_LOCK_MINUTES` (5), `ARGO_ALLOWED_HOSTS` (*) e `ARGO_CORS_ORIGINS` (vazio: só o gateway).

## E-mail de verdade

Troque o Mailpit pelo seu provedor SMTP no `.env` e reinicie o worker. O envio usa o `django.core.mail`, configurado em `MAILERS` a partir destas variáveis:

```
ARGO_SMTP_HOST=smtp.seuprovedor.com
ARGO_SMTP_PORT=587
ARGO_SMTP_USER=usuario
ARGO_SMTP_PASSWORD=senha-de-app
ARGO_SMTP_TLS=true
ARGO_MAIL_FROM=Minha Loja <pedidos@minhaloja.com>
```

```
docker compose up -d worker
```

## Checklist para uso próprio

1. Gere segredos longos e diferentes para `ARGO_JWT_SECRET`, `ARGO_INTERNAL_TOKEN` e `ARGO_DB_PASSWORD` (os scripts já fazem isso).
2. Use `ARGO_SEED=false` em bancos vazios e crie o primeiro administrador pela API ou troque a senha do `admin@argo.dev`.
3. Coloque o gateway atrás de HTTPS (proxy reverso ou balanceador), ajuste `ARGO_PUBLIC_URL` e, se quiser, restrinja `ARGO_ALLOWED_HOSTS`.
4. Configure o SMTP real e o backup do MySQL (veja o PDF do banco de dados).
5. Troque nome e logo em `web/img/`, `web/js/ui.js` e `web/index.html`, se quiser outra marca.

Capítulo 9

# Testes e automação

Os testes rodam sem Docker: o pytest-django cria um banco SQLite para cada serviço, o Celery roda em modo síncrono, os e-mails ficam na caixa de memória do Django e o serviço Java usa H2 em memória no modo MySQL.

## Python

```
cd backend
pip install -r requirements-dev.txt
python -m pytest
```

São 65 testes e a execução falha se a cobertura ficar abaixo de 80% (hoje passa de 90%). Os três serviços rodam no mesmo processo e as chamadas entre eles passam pela aplicação Django de verdade (middleware, autenticação, rotas e erros). Para rodar no MySQL, defina `ARGO_DATABASE_URL`, `ARGO_CATALOG_DATABASE_URL` e `ARGO_ORDERS_DATABASE_URL` (no formato `mysql://usuario:senha@localhost:3306/argo_identity`) antes do `pytest`; o usuário precisa poder criar os bancos `test_*`.

| Arquivo | O que cobre |
|---|---|
| `test_security.py` | JWT válido, expirado, de outro segredo, de outro tipo e papel inválido; rota interna; formato de erro, JSON inválido e método errado; OpenAPI dos três serviços; barramento de eventos |
| `test_identity.py` | Login, bloqueio após 5 erros, cadastro, rotação e reuso de refresh token, logout, troca de senha, gestão de usuários |
| `test_catalog.py` | Busca e filtros, dono do produto, versão otimista, movimentações, baixa tudo ou nada idempotente, devolução, SSE com batimento e mensagens de validação |
| `test_orders.py` | Checkout, validações, catálogo fora do ar, visibilidade por papel, cancelamento, transições, indicadores e histórico de demonstração |
| `test_flow.py` | Integração ponta a ponta: pedido → fila → estoque → e-mail → nota, recusa, cancelamentos, faturamento e SMTP fora do ar |
| `test_mailer.py` | Formatação de dinheiro, e-mail com HTML escapado e envio pelo django.core.mail |

## Java

```
cd billing
mvn -B verify
```

Testes de integração com o servidor real em porta aleatória: emissão idempotente, tributos, total divergente, payload inválido, cancelamento, visibilidade por papel e rejeição de tokens expirados, de outro emissor, de outro tipo, com algoritmo none ou com outro segredo.

## Integração contínua

`.github/workflows/ci.yml` roda a cada envio para `main`, em pull requests e todo dia às 08:17: confere se as migrações do Django estão em dia com os modelos, roda o `check --deploy` do Django, os testes Python com cobertura no SQLite e num MySQL 8.4, os testes Java e a montagem das imagens Docker.

## Atividade diária

`.github/workflows/diario.yml` faz dois commits por dia em `.github/atividade.txt`: o 1º turno às 09:37 e o 2º às 18:37 (Brasília), com um horário reserva para cada turno caso o GitHub atrase o agendamento. O registro é idempotente: se o turno do dia já foi gravado, nada é feito.

1. No repositório, abra **Settings › Secrets and variables › Actions**.
2. Crie `GIT_NAME` com o seu nome de usuário do GitHub.
3. Crie `GIT_EMAIL` com o seu e-mail noreply do GitHub (em **Settings › Emails**), para não expor o e-mail pessoal.
4. Em **Settings › Actions › General**, marque **Read and write permissions**.
5. Para testar, rode o workflow manualmente em **Actions › Atividade diária › Run workflow**.

O CI ignora mudanças só no arquivo de atividade, então os commits automáticos não disparam builds.

Capítulo 10

# Solução de problemas

| Sintoma | Causa provável | O que fazer |
|---|---|---|
| `iniciar.cmd` diz que o Docker não existe | Docker Desktop não instalado ou fechado | Instale ou abra o Docker Desktop e rode de novo |
| Porta 8080 em uso | Outro programa usando a porta | Mude `ARGO_PORT` no `.env` e rode `docker compose up -d` |
| Serviço Python reiniciando | Banco inacessível ou migração com erro | `docker compose logs identity`; o `serve` aplica as migrações antes de subir |
| Pedido fica em Processando | Worker parado ou sem acesso ao Redis | `docker compose logs worker`; reinicie com `docker compose restart worker` |
| Pedido recusado | Outro pedido levou o estoque antes | Comportamento esperado; o motivo aparece no pedido |
| Nota não aparece | Serviço Java subindo ou fora do ar | A fila tenta de novo sozinha; veja `docker compose logs billing` |
| E-mail não chega | SMTP mal configurado | Confira as variáveis `ARGO_SMTP_*` e os logs do worker |
| Volta para a tela de entrada | Página recarregada ou 20 min sem uso | Comportamento esperado da sessão |
| 401 em todas as rotas do Java | Segredo do JWT diferente entre serviços | Use o mesmo `ARGO_JWT_SECRET` para todos |
| 400 Bad Request do Django | Domínio fora de `ARGO_ALLOWED_HOSTS` | Inclua o domínio público na variável |
| Erro 409 ao salvar produto | Alguém alterou antes (versão) | Recarregue a página e salve de novo |
| Quero recomeçar do zero | — | `docker compose down -v` e suba de novo |

Capítulo 11

# Novidades da versão 1.1.0

A versão 1.1.0 corrige falhas encontradas numa revisão completa do código e reforça segurança, confiabilidade e desempenho. As telas continuam com o mesmo visual.

## Correções

- **Gateway que não subia:** o `gateway/nginx.conf` repetia `proxy_read_timeout` na rota do estoque em tempo real, e o Nginx recusava a configuração (`nginx -t` falhava). A rota de Server-Sent Events agora tem os próprios cabeçalhos.
- **Totais grandes:** `orders.total_cents` e `order_items.subtotal_cents` passaram para BIGINT. Pedidos acima de cerca de R$ 21 milhões estouravam o INTEGER do MySQL (migração `0002_totais_bigint_e_indice`).
- **Resumo das notas do Vendedor:** somava a nota inteira, inclusive itens de outras lojas. Agora soma só os itens dele, com os tributos proporcionais, como já faziam os indicadores de pedidos.
- **Preço no cadastro de produto:** digitar `62.90` gravava R$ 6.290,00. Agora vírgula ou ponto decimal são entendidos (`62,90`, `62.90` e `1.234,56`).
- **Cabeçalhos de segurança:** não saíam nos arquivos CSS, JS, fontes e imagens, por causa da regra de herança do `add_header` do Nginx. Ficaram em `gateway/security_headers.conf`, incluído em todas as rotas.

## Confiabilidade

- Se o Redis cair no momento do checkout, o pedido continua gravado e a resposta `202 Accepted` sai normalmente (`transaction.on_commit(..., robust=True)`). Antes a requisição terminava em erro 500.
- A baixa e a devolução de estoque conferem a idempotência de novo depois de travar as linhas: dois workers com o mesmo pedido não geram mais erro 500 nem nova tentativa.
- O produto novo e o estoque inicial são gravados na mesma transação.
- A tela do pedido consulta a situação com espera crescente (de 1,2 s até 10 s) e continua tentando se a rede oscilar.

## Segurança

- As rotas `/api/<serviço>/internal/*` respondem 404 no gateway: só um serviço chama outro, direto pela rede do Docker.
- O gateway descarta o cabeçalho `X-Internal-Token` vindo do navegador.
- Login e cadastro aceitam até 10 tentativas por minuto por IP (excedente recebe HTTP 429), somando-se ao bloqueio de 5 minutos por conta.
- Novos cabeçalhos `Permissions-Policy`, `X-Forwarded-For` e `X-Real-IP`.
- O serviço Java não imprime mais a senha gerada do Spring Security no log, e os erros 405 e 500 seguem o formato `{code, message, fields}`.

## Desempenho

- gzip no gateway para CSS, JS, JSON e SVG: o pacote do Swagger cai de 1,7 MB para 0,45 MB.
- JS e CSS revalidam a cada carga (uma versão nova aparece na hora); fontes e imagens ficam 7 dias em cache.
- Os indicadores ( `GET /orders/stats` ) não carregam mais o histórico de cada pedido, e o novo índice `ix_orders_created` atende ao filtro por período.

## Operação

- `GET /healthz` no gateway e `HEALTHCHECK` na imagem do Nginx.
- Erros do próprio gateway (429, 502, 503 e 504) saem em JSON, e a interface mostra uma mensagem clara em vez de um erro genérico.
- Versão 1.1.0 em todos os serviços. O Java lê a versão do `pom.xml` no `GET /health`.
- Usuários: desativar uma conta pede confirmação.

## Testes

| Suíte | 1.0.0 | 1.1.0 | Novos casos |
|---|---|---|---|
| Python (pytest) | 65 | 67 | fila fora do ar no checkout; pedido com total acima do limite do INTEGER |
| Java (JUnit) | 9 | 11 | resumo do Vendedor só com os itens dele; erros 405 no formato padrão e versão no health |

## Atualizar uma instalação 1.0.0

```
git pull
docker compose up -d --build
```

As migrações do Django rodam sozinhas quando os serviços sobem. Os dados existentes são mantidos.